

HERANÇA DE PROCESSOS: UM ESTUDO PRÁTICO SOBRE CONCORRÊNCIA

Samir de Sousa Herculano

Instituto Federal de Educação, Ciência e Tecnologia do Ceará
samir.herculano04@aluno.ifce.edu.br

RESUMO

Sistemas Operacionais modernos fazem uso de escalonadores preemptivos complexos, levantando dúvidas sobre determinismo na execução de processos concorrentes, este artigo investiga as relações entre processos pai/filho e analisa sua influência na ordem de finalização.

1. INTRODUÇÃO

1.1 Discussão do Problema

Um processo pai cria vários processos filhos em uma determinada ordem. Como os filhos são criados sequencialmente, pode parecer razoável supor que eles também serão executados e terminarão nessa mesma ordem, entretanto, o comportamento observado de escalonadores aponta um problema nessa afirmação, visto que parte-se do pressuposto que algoritmos de escalonamento demonstram comportamento determinístico que pode ser replicado.

1.2 Objetivo

Este artigo tem como objetivo aferir e refutar hipótese acerca da previsibilidade ou natureza determinística dos algoritmos de escalonamento, tendo como base o *kernel* Linux e suas camadas de abstração.

1.3 Hipótese

Supõe-se que a ordem em que os processos filhos são criados influencia a ordem em que eles serão executados e terminarão. Consequentemente, ao utilizar `wait()`, o processo pai tenderá a receber os filhos na mesma ordem em que foram criados.

1.4 Metodologia

Utilizamos um sistema Linux fazendo uso do último *kernel* LTS versão 6.18.43, considerado *stable*, o algoritmo de escalonamento utilizado foi o EEVDF (*Earliest Eligible Virtual Deadline First*), padrão desde o *kernel* versão 6.6. Fora escrito um *script bash* para fins de reprodutibilidade, sendo este sempre executado a partir de um *cold boot* como

meio de evitar interferências de outros programas abertos. O código cria múltiplos processos através de um processo pai com chamadas de sistema `fork()`, numerando do 1 ao 8, segundo a hipótese, devemos obter um resultado final sequencial ao serem finalizados.

2. ANTECEDENTES

- **Modelo de criação e hierarquia:** Em sistemas aderentes ao padrão POSIX, novos processos emergem da duplicação de processos preexistentes (via chamadas como `fork`), estabelecendo uma topologia estrita de pai e filho. O processo filho herda cópias dos espaços de endereçamento, variáveis de ambiente e descritores de arquivos abertos.
- **Políticas de Escalonamento:** O escalonador aloca os ciclos do processador entre os processos no estado "pronto". Algoritmos preemptivos contemporâneos (como o *Earliest Eligible Virtual Deadline First* - EEVDF) modelam o tempo de execução para garantir distribuição equitativa e minimizar a latência de processos interativos.
- **Preempção:** Atributos herdados no momento da criação do processo (classe de escalonamento, afinidade de CPU e valores de prioridade estática/dinâmica) determinam o posicionamento inicial do processo filho na fila do escalonador e a sua respectiva fatia de tempo.

Em sistemas operacionais multiprogramados contemporâneos, a arquitetura de gerenciamento de tarefas fundamenta-se no modelo de herança de processos. A criação de um novo processo ocorre mediante a clonagem do estado do processo gerador, transferindo ao processo filho recursos críticos, tais como descritores de arquivos, limites de alocação e credenciais de segurança. Para mitigar o impacto no desempenho global, técnicas de gerenciamento de memória baseadas em *Copy-on-Write* (CoW) são amplamente adotadas, permitindo o compartilhamento lógico do espaço de endereçamento até que ocorra uma instrução de escrita por parte de um dos processos envolvidos.

Indissociável do mecanismo de herança, o ciclo de vida operacional da tarefa é regido pelas políticas de escalonamento da CPU. A transição contínua entre os estados de prontidão e execução exige algoritmos capazes de evitar a inanição (*starvation*) e maximizar a responsividade global



do sistema. A herança atua como um fator determinante nesta dinâmica, visto que o processo filho assimila parâmetros de escalonamento de seu progenitor — incluindo níveis de prioridade e restrições de afinidade em arquiteturas multiprocessadas. Consequentemente, a eficácia da multiplexação da CPU depende da capacidade do escalonador de balancear os atributos herdados com o comportamento dinâmico da tarefa, ajustando as fatias de tempo em resposta a eventos preemptivos e requisições de entrada e saída.

3. TESTES E RESULTADO

A saída apresentada refuta categoricamente a hipótese. O *script* em *Bash* gera um *CPU burst* idêntico em múltiplos núcleos provocando uma *racing condition*, provocando o resultado visto em [1]

```
18:33
cd Documentos/Coding/Sistemas\Operacionais\Artigo\61/
bash -rfile /etc/profile -c ./processosbash.sh
[Pai] Iniciado (PID: 4173)
[Filho 1] Iniciado (PID: 4174)
[Filho 2] Iniciado (PID: 4175)
[Filho 3] Iniciado (PID: 4176)
[Filho 4] Iniciado (PID: 4177)
[Filho 5] Iniciado (PID: 4178)
[Filho 6] Iniciado (PID: 4179)
[Filho 7] Iniciado (PID: 4180)
Ordem de criação (PIDs): 4174 4175 4176 4177 4178 4179 4180 4181
Aguardando filhos...
[Filho 8] Iniciado (PID: 4181)
[Pai] Término capturado -> PID: 4179 | Código (ID Original): 6
[Pai] Término capturado -> PID: 4175 | Código (ID Original): 2
[Pai] Término capturado -> PID: 4180 | Código (ID Original): 7
[Pai] Término capturado -> PID: 4178 | Código (ID Original): 3
[Pai] Término capturado -> PID: 4178 | Código (ID Original): 5
[Pai] Término capturado -> PID: 4181 | Código (ID Original): 8
[Pai] Término capturado -> PID: 4174 | Código (ID Original): 1
[Pai] Término capturado -> PID: 4177 | Código (ID Original): 4
```

Figura 1. Execução do *script*, exemplifica a criação e finalização de processos.

Observa-se que as mensagens de inicialização dos Filhos 1 a 7 aparecem antes do pai imprimir a lista de PIDs. No entanto, o Filho 8 aparece após o pai já ter impresso a mensagem de aguardo. O comportamento pode ser explicado pelo fato da criação de um processo se dar pela *syscall* *fork()*. O *kernel* Linux aceita o pedido e gera o PID 4173, o Pai continua rodando imediatamente e imprime suas mensagens, enquanto o Filho 8 ainda está sendo alocado na memória ou aguardando um núcleo livre para imprimir na tela, demonstrando um comportamento assíncrono. Adicionalmente, ao fazer uso de *wait(NULL)* capturou-se o momento exato em que cada processo foi finalizado, assim desprovando a premissa inicial de que a saída seria sequencial (do 1 ao 8), que implicava comportamento determinístico no algoritmo escalonamento.

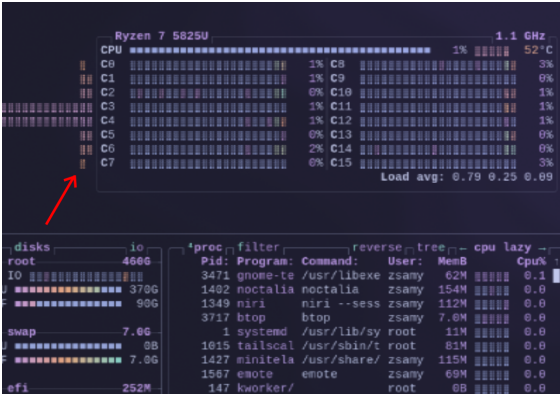


Figura 2. Por meio do gráfico exibido, é possível notar o pico de atividade na CPU gerado pelo *burst*.

4. REFERÊNCIAS

[1] B. Rice, “The Tragedy of systemd,” na *Universidade de Canterbury*, *linux.conf.au*, (Christchurch, Nova Zelândia), 2019.

[2] A. S. Tanenbaum e H. Bos *Sistemas Operacionais Modernos*. São Paulo, Brasil: Bookman, 2024.

[3] A. S. Tanenbaum e A. S. Woodhull *Sistemas Operacionais: Projeto e Implementação*. São Paulo, Brasil: Bookman, 2008.